```
```

```
```

```
```

```
```

```
```

```
# Part 1 — Core Implementation
# Target: Python 3.11+ / CPython
# Dependencies: numpy, scipy, pandas, matplotlib, networkx, sympy

# ============================================================
# 1. Foundations: Sets, Logic, and Proof Helpers
# ============================================================

from __future__ import annotations
from typing import Iterable, Callable, Hashable
import itertools
import math

def powerset(iterable: Iterable) -> list[tuple]:
    """Return all subsets of the input as tuples."""
    s = list(iterable)
    return [tuple(c) for r in range(len(s) + 1)
            for c in itertools.combinations(s, r)]

assert powerset("ab") == [(), ('a',), ('b',), ('a', 'b')]

def cartesian_power(A: Iterable, n: int) -> list[tuple]:
    """A^n as a list of tuples."""
    return list(itertools.product(A, repeat=n))

assert len(cartesian_power({0, 1}, 3)) == 8

def is_relation(R: Iterable[tuple], A: Iterable, B: Iterable) -> bool:
    """Check that every pair in R lies in A x B."""
    A, B = set(A), set(B)
    return all(a in A and b in B for (a, b) in R)

# ------------------------------------------------------------
# 1.1 Truth tables and tautology checking
# ------------------------------------------------------------

def truth_table(f: Callable[..., bool], n: int):
    """Yield (inputs, output) for every assignment of n booleans."""
    for bits in itertools.product([False, True], repeat=n):
        yield bits, f(*bits)

def is_tautology(f: Callable[..., bool], n: int) -> bool:
    return all(out for _, out in truth_table(f, n))

def is_contradiction(f: Callable[..., bool], n: int) -> bool:
    return not any(out for _, out in truth_table(f, n))

# De Morgan: not(p and q) == (not p) or (not q)
assert is_tautology(lambda p, q: (not (p and q)) == ((not p) or (not q)), 2)

# ------------------------------------------------------------
# 1.2 Equivalence relations and partitions
# ------------------------------------------------------------

def is_equivalence(R: Iterable[tuple], A: Iterable) -> bool:
    A = set(A)
    R = set(R)
    if not is_relation(R, A, A):
        return False
    reflexive = all((a, a) in R for a in A)
    symmetric = all((b, a) in R for (a, b) in R)
    transitive = all((a, d) in R
                     for (a, b) in R for (c, d) in R if b == c)
    return reflexive and symmetric and transitive

def partition_to_relation(parts: Iterable[Iterable]) -> set[tuple]:
    return {(a, b) for part in parts for a in part for b in part}

def relation_to_partition(R: Iterable[tuple], A: Iterable) -> list[set]:
    A = set(A)
    R = set(R)
    blocks: list[set] = []
    for a in A:
        for block in blocks:
            if any((a, x) in R and (x, a) in R for x in block):
                block.add(a)
                break
        else:
            blocks.append({a})
    return blocks

# ============================================================
# 2. Number Theory: Primes, GCD, Modular Arithmetic
# ============================================================

def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return abs(a)

def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
    """Return (g, x, y) with a*x + b*y == g == gcd(a, b)."""
    if b == 0:
        return (a, 1, 0) if a >= 0 else (-a, -1, 0)
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def lcm(a: int, b: int) -> int:
    return abs(a // gcd(a, b) * b) if a and b else 0

def sieve(limit: int) -> list[int]:
    """Sieve of Eratosthenes: primes <= limit."""
    if limit < 2:
        return []
    is_prime = bytearray([1]) * (limit + 1)
    is_prime[0:2] = b"\x00\x00"
    for p in range(2, int(limit ** 0.5) + 1):
        if is_prime[p]:
            is_prime[p * p : limit + 1 : p] = bytearray(
                len(range(p * p, limit + 1, p)))
    return [i for i, flag in enumerate(is_prime) if flag]

assert sieve(30) == [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

def is_prime(n: int) -> bool:
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def prime_factorization(n: int) -> dict[int, int]:
    """Return {prime: exponent} for n >= 1."""
    factors: dict[int, int] = {}
    d = 2
    while d * d <= n:
        while n % d == 0:
            factors[d] = factors.get(d, 0) + 1
            n //= d
        d += 1 if d == 2 else 2
    if n > 1:
        factors[n] = factors.get(n, 0) + 1
    return factors

assert prime_factorization(360) == {2: 3, 3: 2, 5: 1}

def mod_inverse(a: int, m: int) -> int:
    """Multiplicative inverse of a mod m, if it exists."""
    g, x, _ = extended_gcd(a % m, m)
    if g != 1:
        raise ValueError(f"{a} has no inverse mod {m}")
    return x % m

def mod_pow(base: int, exp: int, m: int) -> int:
    """Fast modular exponentiation (exp may be negative)."""
    if exp < 0:
        base = mod_inverse(base, m)
        exp = -exp
    result = 1
    base %= m
    while exp:
        if exp & 1:
            result = result * base % m
        base = base * base % m
        exp >>= 1
    return result

assert mod_pow(3, 200, 7) == pow(3, 200, 7)

def crt(residues: list[int], moduli: list[int]) -> int:
    """Chinese Remainder Theorem for pairwise-coprime moduli."""
    assert len(residues) == len(moduli)
    M = 1
    for m in moduli:
        M *= m
    x = 0
    for r, m in zip(residues, moduli):
        Mi = M // m
        x += r * Mi * mod_inverse(Mi, m)
    return x % M

assert crt([2, 3, 2], [3, 5, 7]) == 23

# ============================================================
# 3. Combinatorics
# ============================================================

def factorial(n: int) -> int:
    return math.factorial(n)

def binomial(n: int, k: int) -> int:
    if k < 0 or k > n:
        return 0
    return factorial(n) // (factorial(k) * factorial(n - k))

def multinomial(ks: list[int]) -> int:
    n = sum(ks)
    out = factorial(n)
    for k in ks:
        out //= factorial(k)
    return out

def stirling2(n: int, k: int) -> int:
    """Stirling numbers of the second kind S(n, k)."""
    if n == k == 0:
        return 1
    if k == 0 or k > n:
        return 0
    return k * stirling2(n - 1, k) + stirling2(n - 1, k - 1)

def bell(n: int) -> int:
    return sum(stirling2(n, k) for k in range(n + 1))

assert [bell(i) for i in range(6)] == [1, 1, 2, 5, 15, 52]

def derangements(n: int) -> int:
    return round(factorial(n) / math.e)

assert derangements(4) == 9

def integer_partitions(n: int, max_part: int | None = None):
    """Yield partitions of n as non-increasing tuples."""
    if max_part is None or max_part > n:
        max_part = n
    if n == 0:
        yield ()
        return
    for first in range(max_part, 0, -1):
        for rest in integer_partitions(n - first, first):
            yield (first,) + rest

assert len(list(integer_partitions(5))) == 7

# ============================================================
# 4. Graph Theory
# ============================================================

import networkx as nx

def random_graph(n: int, p: float, seed: int = 0) -> nx.Graph:
    return nx.gnp_random_graph(n, p, seed=seed)

def is_bipartite_coloring(G: nx.Graph) -> dict | None:
    """Return a 2-coloring dict, or None if G is not bipartite."""
    try:
        return nx.bipartite.color(G)
    except nx.NetworkXError:
        return None

def chromatic_number_upper_bound(G: nx.Graph) -> int:
    """Greedy coloring upper bound (Delta + 1)."""
    return max(nx.greedy_color(G, strategy="largest_first").values()) + 1

def shortest_path_or_none(G: nx.Graph, u, v):
    try:
        return nx.shortest_path(G, u, v)
    except nx.NetworkXNoPath:
        return None

def eulerian_circuit_or_none(G: nx.Graph):
    if nx.is_eulerian(G):
        return list(nx.eulerian_circuit(G))
    return None

def minimum_spanning_tree_weight(G: nx.Graph) -> float:
    T = nx.minimum_spanning_tree(G)
    return sum(d["weight"] for _, _, d in T.edges(data=True))

def max_flow(G: nx.DiGraph, s, t) -> float:
    value, _ = nx.maximum_flow(G, s, t)
    return value

def graph_isomorphic(G1: nx.Graph, G2: nx.Graph) -> bool:
    return nx.is_isomorphic(G1, G2)

def planarity(G: nx.Graph) -> bool:
    return nx.check_planarity(G)[0]

def clique_number(G: nx.Graph) -> int:
    return max(len(c) for c in nx.find_cliques(G))

def dominating_set_greedy(G: nx.Graph) -> set:
    return nx.dominating_set(G)

# ============================================================
# 5. Linear Algebra
# ============================================================

import numpy as np

def rref(A: np.ndarray, tol: float = 1e-12) -> tuple[np.ndarray, list[int]]:
    """Reduced row echelon form and pivot columns."""
    A = A.astype(float).copy()
    rows, cols = A.shape
    pivots = []
    r = 0
    for c in range(cols):
        if r >= rows:
            break
        pivot = r + np.argmax(np.abs(A[r:, c]))
        if abs(A[pivot, c]) < tol:
            continue
        A[[r, pivot]] = A[[pivot, r]]
        A[r] /= A[r, c]
        for i in range(rows):
            if i != r:
                A[i] -= A[i, c] * A[r]
        pivots.append(c)
        r += 1
    return A, pivots

def rank(A: np.ndarray) -> int:
    return len(rref(A)[1])

def nullspace(A: np.ndarray, tol: float = 1e-12) -> np.ndarray:
    """Basis for the nullspace, as columns."""
    R, pivots = rref(A, tol)
    cols = A.shape[1]
    free = [c for c in range(cols) if c not in pivots]
    basis = np.zeros((cols, len(free)))
    for j, f in enumerate(free):
        basis[f, j] = 1
        for i, p in enumerate(pivots):
            basis[p, j] = -R[i, f]
    return basis

def solve_or_none(A: np.ndarray, b: np.ndarray):
    try:
        return np.linalg.solve(A, b)
    except np.linalg.LinAlgError:
        return None

def eigen_sorted(A: np.ndarray):
    """Eigenvalues and eigenvectors, sorted by |lambda| descending."""
    vals, vecs = np.linalg.eig(A)
    order = np.argsort(-np.abs(vals))
    return vals[order], vecs[:, order]

def power_iteration(A: np.ndarray, iters: int = 1000, seed: int = 0):
    rng = np.random.default_rng(seed)
    v = rng.standard_normal(A.shape[0])
    for _ in range(iters):
        v = A @ v
        v /= np.linalg.norm(v)
    return v @ A @ v, v

def svd_reconstruct(A: np.ndarray, k: int | None = None) -> np.ndarray:
    U, s, Vt = np.linalg.svd(A, full_matrices=False)
    if k is not None:
        U, s, Vt = U[:, :k], s[:k], Vt[:k]
    return U @ np.diag(s) @ Vt

def pseudoinverse(A: np.ndarray) -> np.ndarray:
    return np.linalg.pinv(A)

def condition_number(A: np.ndarray) -> float:
    return np.linalg.cond(A)

def matrix_exp(A: np.ndarray, terms: int = 30) -> np.ndarray:
    """exp(A) via truncated Taylor series."""
    out = np.eye(A.shape[0])
    term = np.eye(A.shape[0])
    for k in range(1, terms):
        term = term @ A / k
        out = out + term
    return out

def kronecker(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    return np.kron(A, B)

def cholesky_or_none(A: np.ndarray):
    try:
        return np.linalg.cholesky(A)
    except np.linalg.LinAlgError:
        return None

def qr_decompose(A: np.ndarray):
    return np.linalg.qr(A)

def lu_decompose(A: np.ndarray):
    from scipy.linalg import lu
    return lu(A)

def least_squares(A: np.ndarray, b: np.ndarray):
    x, residuals, rank_, sv = np.linalg.lstsq(A, b, rcond=None)
    return x, residuals, rank_, sv

def projection_matrix(A: np.ndarray) -> np.ndarray:
    return A @ np.linalg.pinv(A)

def gram_schmidt(A: np.ndarray) -> np.ndarray:
    Q, _ = np.linalg.qr(A)
    return Q

def hilbert(n: int) -> np.ndarray:
    return np.array([[1 / (i + j + 1) for j in range(n)] for i in range(n)])

def vandermonde(x: np.ndarray, n: int) -> np.ndarray:
    return np.vander(x, n, increasing=True)

def companion(coeffs: list[float]) -> np.ndarray:
    return np.array(np.polynomial.polynomial.polycompanion(coeffs))

def roots_of_polynomial(coeffs: list[float]) -> np.ndarray:
    return np.roots(coeffs[::-1])

def trace(A: np.ndarray) -> float:
    return float(np.trace(A))

def determinant(A: np.ndarray) -> float:
    return float(np.linalg.det(A))

def frobenius_norm(A: np.ndarray) -> float:
    return float(np.linalg.norm(A, "fro"))

def spectral_norm(A: np.ndarray) -> float:
    return float(np.linalg.norm(A, 2))

def nuclear_norm(A: np.ndarray) -> float:
    return float(np.linalg.norm(A, "nuc"))

def is_orthogonal(A: np.ndarray, tol: float = 1e-10) -> bool:
    return np.allclose(A @ A.T, np.eye(A.shape[0]), atol=tol)

def is_symmetric(A: np.ndarray, tol: float = 1e-10) -> bool:
    return np.allclose(A, A.T, atol=tol)

def is_positive_definite(A: np.ndarray) -> bool:
    return cholesky_or_none(A) is not None

def random_orthogonal(n: int, seed: int = 0) -> np.ndarray:
    rng = np.random.default_rng(seed)
    Q, R = np.linalg.qr(rng.standard_normal((n, n)))
    return Q * np.sign(np.diag(R))

def random_spd(n: int, seed: int = 0) -> np.ndarray:
    rng = np.random.default_rng(seed)
    A = rng.standard_normal((n, n))
    return A @ A.T + n * np.eye(n)

# ============================================================
# 6. Probability and Statistics
# ============================================================

import pandas as pd
from scipy import stats

def bernoulli_pmf(k: int, p: float) -> float:
    return p if k == 1 else 1 - p

def binomial_pmf(k: int, n: int, p: float) -> float:
    return binomial(n, k) * p ** k * (1 - p) ** (n - k)

def poisson_pmf(k: int, lam: float) -> float:
    return lam ** k * math.exp(-lam) / factorial(k)

def geometric_pmf(k: int, p: float) -> float:
    return (1 - p) ** (k - 1) * p

def normal_pdf(x: float, mu: float = 0, sigma: float = 1) -> float:
    return math.exp(-((x - mu) / sigma) ** 2 / 2) / (sigma * math.sqrt(2 * math.pi))

def normal_cdf(x: float, mu: float = 0, sigma: float = 1) -> float:
    return 0.5 * (1 + math.erf((x - mu) / (sigma * math.sqrt(2))))

def expectation(values: Iterable[float], probs: Iterable[float]) -> float:
    return sum(v * p for v, p in zip(values, probs))

def variance(values: Iterable[float], probs: Iterable[float]) -> float:
    mu = expectation(values, probs)
    return sum((v - mu) ** 2 * p for v, p in zip(values, probs))

def covariance(xs, ys, probs) -> float:
    mx = expectation(xs, probs)
    my = expectation(ys, probs)
    return sum((x - mx) * (y - my) * p for x, y, p in zip(xs, ys, probs))

def correlation(xs, ys, probs) -> float:
    return covariance(xs, ys, probs) / math.sqrt(variance(xs, probs) * variance(ys, probs))

def sample_mean(xs: Iterable[float]) -> float:
    xs = list(xs)
    return sum(xs) / len(xs)

def sample_variance(xs: Iterable[float], ddof: int = 1) -> float:
    xs = list(xs)
    m = sample_mean(xs)
    return sum((x - m) ** 2 for x in xs) / (len(xs) - ddof)

def sample_std(xs: Iterable[float], ddof: int = 1) -> float:
    return math.sqrt(sample_variance(xs, ddof))

def standard_error(xs: Iterable[float]) -> float:
    xs = list(xs)
    return sample_std(xs) / math.sqrt(len(xs))

def z_score(x: float, mu: float, sigma: float) -> float:
    return (x - mu) / sigma

def t_test_one_sample(xs: Iterable[float], mu0: float):
    return stats.ttest_1samp(list(xs), mu0)

def t_test_two_sample(xs: Iterable[float], ys: Iterable[float]):
    return stats.ttest_ind(list(xs), list(ys), equal_var=False)

def chi_square_test(observed: Iterable[float], expected: Iterable[float]):
    return stats.chisquare(list(observed), list(expected))

def anova_one_way(*groups):
    return stats.f_oneway(*groups)

def pearson_r(xs: Iterable[float], ys: Iterable[float]):
    return stats.pearsonr(list(xs), list(ys))

def spearman_rho(xs: Iterable[float], ys: Iterable[float]):
    return stats.spearmanr(list(xs), list(ys))

def kolmogorov_smirnov(xs: Iterable[float], cdf: Callable[[float], float]):
    return stats.kstest(list(xs), cdf)

def bootstrap_mean_ci(xs: Iterable[float], n_resamples: int = 2000,
                      alpha: float = 0.05, seed: int = 0):
    rng = np.random.default_rng(seed)
    xs = np.asarray(list(xs))
    means = np.array([rng.choice(xs, size=len(xs), replace=True).mean()
                      for _ in range(n_resamples)])
    return np.quantile(means, [alpha / 2, 1 - alpha / 2])

def permutation_test(xs, ys, n_resamples: int = 2000, seed: int = 0):
    rng = np.random.default_rng(seed)
    xs, ys = np.asarray(xs), np.asarray(ys)
    observed = xs.mean() - ys.mean()
    pooled = np.concatenate([xs, ys])
    count = 0
    for _ in range(n_resamples):
        rng.shuffle(pooled)
        if pooled[:len(xs)].mean() - pooled[len(xs):].mean() >= observed:
            count += 1
    return count / n_resamples

def ecdf(xs: Iterable[float]):
    xs = sorted(xs)
    n = len(xs)
    return lambda x: sum(1 for v in xs if v <= x) / n

def histogram_bins(xs: Iterable[float], bins: int = 10):
    return np.histogram(list(xs), bins=bins)

def quantile(xs: Iterable[float], q: float) -> float:
    return float(np.quantile(list(xs), q))

def iqr(xs: Iterable[float]) -> float:
    return quantile(xs, 0.75) - quantile(xs, 0.25)

def skewness(xs: Iterable[float]) -> float:
    return stats.skew(list(xs))

def kurtosis(xs: Iterable[float]) -> float:
    return stats.kurtosis(list(xs))

def describe(xs: Iterable[float]) -> pd.Series:
    return pd.Series(list(xs)).describe()

# ============================================================
# 7. Optimization
# ============================================================

from scipy.optimize import minimize, linprog, root_scalar

def gradient_descent(f, grad, x0, lr: float = 0.1, steps: int = 1000):
    x = np.asarray(x0, dtype=float)
    for _ in range(steps):
        x = x - lr * grad(x)
    return x

def newton_1d(f, df, x0: float, tol: float = 1e-10, max_iter: int = 50):
    x = x0
    for _ in range(max_iter):
        step = f(x) / df(x)
        x -= step
        if abs(step) < tol:
            break
    return x

def bisection(f, a: float, b: float, tol: float = 1e-10):
    fa, fb = f(a), f(b)
    if fa * fb > 0:
        raise ValueError("f(a) and f(b) must have opposite signs")
    while b - a > tol:
        m = (a + b) / 2
        fm = f(m)
        if fa * fm <= 0:
            b, fb = m, fm
        else:
            a, fa = m, fm
    return (a + b) / 2

def golden_section_min(f, a: float, b: float, tol: float = 1e-8):
    phi = (1 + 5 ** 0.5) / 2
    c = b - (b - a) / phi
    d = a + (b - a) / phi
    while abs(b - a) > tol:
        if f(c) < f(d):
            b = d
        else:
            a = c
        c = b - (b - a) / phi
        d = a + (b - a) / phi
    return (a + b) / 2

def nelder_mead(f, x0):
    return minimize(f, x0, method="Nelder-Mead").x

def bfgs(f, x0):
    return minimize(f, x0, method="BFGS").x

def linear_program(c, A_ub=None, b_ub=None, A_eq=None, b_eq=None, bounds=None):
    return linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds)

def root_of(f, bracket: tuple[float, float]):
    return root_scalar(f, bracket=bracket).root

def numerical_gradient(f, x, h: float = 1e-6):
    x = np.asarray(x, dtype=float)
    g = np.zeros_like(x)
    for i in range(len(x)):
        e = np.zeros_like(x)
        e[i] = h
        g[i] = (f(x + e) - f(x - e)) / (2 * h)
    return g

def numerical_hessian(f, x, h: float = 1e-4):
    x = np.asarray(x, dtype=float)
    n = len(x)
    H = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            ei = np.zeros(n); ei[i] = h
            ej = np.zeros(n); ej[j] = h
            H[i, j] = (f(x + ei + ej) - f(x + ei - ej)
                       - f(x - ei + ej) + f(x - ei - ej)) / (4 * h * h)
    return H

# ============================================================
# 8. Numerical Methods
# ============================================================

def trapezoid(f, a: float, b: float, n: int = 1000):
    xs = np.linspace(a, b, n + 1)
    ys = f(xs)
    return (b - a) / n * (ys[0] / 2 + ys[1:-1].sum() + ys[-1] / 2)

def simpson(f, a: float, b: float, n: int = 1000):
    if n % 2:
        n += 1
    xs = np.linspace(a, b, n + 1)
    ys = f(xs)
    return (b - a) / n / 3 * (ys[0] + 4 * ys[1:-1:2].sum() + 2 * ys[2:-1:2].sum() + ys[-1])

def adaptive_quad(f, a: float, b: float):
    from scipy.integrate import quad
    return quad(f, a, b)[0]

def euler_ode(f, y0, t0: float, t1: float, steps: int = 1000):
    y = np.asarray(y0, dtype=float)
    t = t0
    dt = (t1 - t0) / steps
    for _ in range(steps):
        y = y + dt * f(t, y)
        t += dt
    return y

def rk4_ode(f, y0, t0: float, t1: float, steps: int = 1000):
    y = np.asarray(y0, dtype=float)
    t = t0
    dt = (t1 - t0) / steps
    for _ in range(steps):
        k1 = f(t, y)
        k2 = f(t + dt / 2, y + dt / 2 * k1)
        k3 = f(t + dt / 2, y + dt / 2 * k2)
        k4 = f(t + dt, y + dt * k3)
        y = y + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
        t += dt
    return y

def finite_difference_jacobian(f, x, h: float = 1e-6):
    x = np.asarray(x, dtype=float)
    fx = f(x)
    J = np.zeros((len(fx), len(x)))
    for i in range(len(x)):
        e = np.zeros_like(x)
        e[i] = h
        J[:, i] = (f(x + e) - f(x - e)) / (2 * h)
    return J

def fixed_point(g, x0: float, tol: float = 1e-10, max_iter: int = 200):
    x = x0
    for _ in range(max_iter):
        nx = g(x)
        if abs(nx - x) < tol:
            return nx
        x = nx
    raise RuntimeError("fixed_point did not converge")

def newton_system(F, J, x0, tol: float = 1e-10, max_iter: int = 50):
    x = np.asarray(x0, dtype=float)
    for _ in range(max_iter):
        step = np.linalg.solve(J(x), F(x))
        x -= step
        if np.linalg.norm(step) < tol:
            break
    return x

def interp_linear(xs, ys, x):
    return np.interp(x, xs, ys)

def cubic_spline(xs, ys):
    from scipy.interpolate import CubicSpline
    return CubicSpline(xs, ys)

def lagrange_interp(xs, ys, x):
    xs = np.asarray(xs, dtype=float)
    ys = np.asarray(ys, dtype=float)
    total = 0.0
    for i in range(len(xs)):
        term = ys[i]
        for j in range(len(xs)):
            if i != j:
                term *= (x - xs[j]) / (xs[i] - xs[j])
        total += term
    return total

def fft_convolve(a, b):
    from scipy.signal import fftconvolve
    return fftconvolve(a, b)

def dft(x):
    return np.fft.fft(x)

def idft(X):
    return np.fft.ifft(X)

# ============================================================
# 9. Machine Learning Primitives
# ============================================================

def one_hot(labels, num_classes: int | None = None):
    labels = np.asarray(labels)
    if num_classes is None:
        num_classes = labels.max() + 1
    out = np.zeros((len(labels), num_classes))
    out[np.arange(len(labels)), labels] = 1
    return out

def softmax(z, axis: int = -1):
    z = z - z.max(axis=axis, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

def log_softmax(z, axis: int = -1):
    z = z - z.max(axis=axis, keepdims=True)
    return z - np.log(np.exp(z).sum(axis=axis, keepdims=True))

def cross_entropy(p, q, eps: float = 1e-12):
    q = np.clip(q, eps, 1 - eps)
    return -(p * np.log(q)).sum()

def kl_divergence(p, q, eps: float = 1e-12):
    p = np.clip(p, eps, 1)
    q = np.clip(q, eps, 1)
    return (p * np.log(p / q)).sum()

def mse(y_true, y_pred):
    return ((y_true - y_pred) ** 2).mean()

def mae(y_true, y_pred):
    return np.abs(y_true - y_pred).mean()

def r_squared(y_true, y_pred):
    ss_res = ((y_true - y_pred) ** 2).sum()
    ss_tot = ((y_true - y_true.mean()) ** 2).sum()
    return 1 - ss_res / ss_tot

def accuracy(y_true, y_pred):
    return (y_true == y_pred).mean()

def confusion_matrix(y_true, y_pred, num_classes: int | None = None):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    if num_classes is None:
        num_classes = max(y_true.max(), y_pred.max()) + 1
    cm = np.zeros((num_classes, num_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm

def precision_recall_f1(y_true, y_pred, positive=1):
    tp = ((y_pred == positive) & (y_true == positive)).sum()
    fp = ((y_pred == positive) & (y_true != positive)).sum()
    fn = ((y_pred != positive) & (y_true == positive)).sum()
    prec = tp / (tp + fp) if tp + fp else 0.0
    rec = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * prec * rec / (prec + rec) if prec + rec else 0.0
    return prec, rec, f1

def roc_auc(y_true, scores):
    from sklearn.metrics import roc_auc_score
    return roc_auc_score(y_true, scores)

def train_test_split(X, y, test_frac: float = 0.2, seed: int = 0):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(X))
    cut = int(len(X) * (1 - test_frac))
    return X[idx[:cut]], X[idx[cut:]], y[idx[:cut]], y[idx[cut:]]

def standardize(X):
    mu = X.mean(axis=0)
    sigma = X.std(axis=0)
    sigma[sigma == 0] = 1
    return (X - mu) / sigma, mu, sigma

def min_max_scale(X):
    lo = X.min(axis=0)
    hi = X.max(axis=0)
    span = hi - lo
    span[span == 0] = 1
    return (X - lo) / span, lo, hi

def pca(X, k: int):
    Xc = X - X.mean(axis=0)
    U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
    return Xc @ Vt[:k].T, Vt[:k], s[:k] ** 2 / (len(X) - 1)

def kmeans(X, k: int, iters: int = 100, seed: int = 0):
    rng = np.random.default_rng(seed)
    centers = X[rng.choice(len(X), k, replace=False)]
    for _ in range(iters):
        d = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)
        labels = d.argmin(axis=1)
        new = np.array([X[labels == j].mean(axis=0) if (labels == j).any()
                        else centers[j] for j in range(k)])
        if np.allclose(new, centers):
            break
        centers = new
    return labels, centers

def linear_regression_fit(X, y):
    Xb = np.hstack([np.ones((len(X), 1)), X])
    w, *_ = np.linalg.lstsq(Xb, y, rcond=None)
    return w

def linear_regression_predict(w, X):
    Xb = np.hstack([np.ones((len(X), 1)), X])
    return Xb @ w

def ridge_fit(X, y, lam: float = 1.0):
    Xb = np.hstack([np.ones((len(X), 1)), X])
    I = np.eye(Xb.shape[1])
    I[0, 0] = 0
    return np.linalg.solve(Xb.T @ Xb + lam * I, Xb.T @ y)

def logistic_fit(X, y, lr: float = 0.1, steps: int = 2000):
    Xb = np.hstack([np.ones((len(X), 1)), X])
    w = np.zeros(Xb.shape[1])
    for _ in range(steps):
        p = 1 / (1 + np.exp(-Xb @ w))
        w -= lr * Xb.T @ (p - y) / len(y)
    return w

def logistic_predict(w, X):
    Xb = np.hstack([np.ones((len(X), 1)), X])
    return (1 / (1 + np.exp(-Xb @ w)) >= 0.5).astype(int)

def knn_predict(X_train, y_train, X_test, k: int = 5):
    d = ((X_test[:, None, :] - X_train[None, :, :]) ** 2).sum(axis=2)
    idx = d.argsort(axis=1)[:, :k]
    votes = y_train[idx]
    return np.array([np.bincount(row).argmax() for row in votes])

def decision_tree_fit(X, y, max_depth: int = 5):
    from sklearn.tree import DecisionTreeClassifier
    return DecisionTreeClassifier(max_depth=max_depth).fit(X, y)

def random_forest_fit(X, y, n_trees: int = 100, seed: int = 0):
    from sklearn.ensemble import RandomForestClassifier
    return RandomForestClassifier(n_estimators=n_trees, random_state=seed).fit(X, y)

def gradient_boosting_fit(X, y, n_trees: int = 100):
    from sklearn.ensemble import GradientBoostingClassifier
    return GradientBoostingClassifier(n_estimators=n_trees).fit(X, y)

def svm_fit(X, y, C: float = 1.0):
    from sklearn.svm import SVC
    return SVC(C=C).fit(X, y)

def naive_bayes_fit(X, y):
    from sklearn.naive_bayes import GaussianNB
    return GaussianNB().fit(X, y)

def cross_validate(model_fit, model_predict, X, y, folds: int = 5, seed: int = 0):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(X))
    parts = np.array_split(idx, folds)
    scores = []
    for i in range(folds):
        test = parts[i]
        train = np.concatenate([parts[j] for j in range(folds) if j != i])
        model = model_fit(X[train], y[train])
        scores.append(accuracy(y[test], model_predict(model, X[test])))
    return scores

def grid_search(model_fit, model_predict, X, y, grid: dict, folds: int = 5):
    import itertools
    keys = list(grid)
    best = (-1, None)
    for values in itertools.product(*grid.values()):
        params = dict(zip(keys, values))
        scores = cross_validate(lambda Xt, yt: model_fit(Xt, yt, **params),
                                model_predict, X, y, folds)
        mean = sum(scores) / len(scores)
        if mean > best[0]:
            best = (mean, params)
    return best

# ============================================================
# 10. Signal Processing
# ============================================================

def moving_average(x, window: int):
    return np.convolve(x, np.ones(window) / window, mode="valid")

def exponential_smoothing(x, alpha: float):
    out = [x[0]]
    for v in x[1:]:
        out.append(alpha * v + (1 - alpha) * out[-1])
    return np.array(out)

def autocorrelation(x, max_lag: int):
    x = x - x.mean()
    return np.array([(x[:len(x) - k] * x[k:]).sum() / (x * x).sum()
                     for k in range(max_lag + 1)])

def low_pass_filter(x, cutoff: float, fs: float, order: int = 4):
    from scipy.signal import butter, filtfilt
    b, a = butter(order, cutoff / (fs / 2), btype="low")
    return filtfilt(b, a, x)

def high_pass_filter(x, cutoff: float, fs: float, order: int = 4):
    from scipy.signal import butter, filtfilt
    b, a = butter(order, cutoff / (fs / 2), btype="high")
    return filtfilt(b, a, x)

def band_pass_filter(x, lo: float, hi: float, fs: float, order: int = 4):
    from scipy.signal import butter, filtfilt
    b, a = butter(order, [lo / (fs / 2), hi / (fs / 2)], btype="band")
    return filtfilt(b, a, x)

def spectrogram(x, fs: float):
    from scipy.signal import spectrogram
    return spectrogram(x, fs)

def hilbert_envelope(x):
    from scipy.signal import hilbert
    return np.abs(hilbert(x))

def zero_crossings(x):
    return ((x[:-1] * x[1:]) < 0).sum()

def peak_detect(x, threshold: float):
    from scipy.signal import find_peaks
    return find_peaks(x, height=threshold)

def resample_signal(x, num: int):
    from scipy.signal import resample
    return resample(x, num)

def cross_correlation(x, y):
    return np.correlate(x, y, mode="full")

def window_hann(n: int):
    return np.hanning(n)

def window_hamming(n: int):
    return np.hamming(n)

def window_blackman(n: int):
    return np.blackman(n)

def rms(x):
    return np.sqrt((x ** 2).mean())

def snr_db(signal, noise):
    return 10 * np.log10((signal ** 2).mean() / (noise ** 2).mean())

def zero_pad(x, n: int):
    return np.pad(x, (0, max(0, n - len(x))))

def unwrap_phase(x):
    return np.unwrap(x)

def instantaneous_frequency(x, fs: float):
    return np.diff(np.unwrap(np.angle(x))) * fs / (2 * np.pi)

def median_filter(x, size: int):
    from scipy.signal import medfilt
    return medfilt(x, size)

def savitzky_golay(x, window: int, poly: int):
    from scipy.signal import savgol_filter
    return savgol_filter(x, window, poly)

def wiener_filter(x, noise_power: float):
    from scipy.signal import wiener
    return wiener(x, noise=noise_power)

def detrend(x):
    from scipy.signal import detrend
    return detrend(x)

def cepstrum(x):
    return np.fft.ifft(np.log(np.abs(np.fft.fft(x)) + 1e-12)).real

def mel_filterbank(n_filters: int, n_fft: int, fs: float):
    from librosa.filters import mel
    return mel(sr=fs, n_fft=n_fft, n_mels=n_filters)

def mfcc(x, fs: float, n_mfcc: int = 13):
    from librosa.feature import mfcc
    return mfcc(y=x, sr=fs, n_mfcc=n_mfcc)

def chroma(x, fs: float):
    from librosa.feature import chroma_stft
    return chroma_stft(y=x, sr=fs)

def onset_detect(x, fs: float):
    from librosa.onset import onset_detect
    return onset_detect(y=x, sr=fs)

def tempo_estimate(x, fs: float):
    from librosa.beat import tempo
    return tempo(y=x, sr=fs)

def pitch_track(x, fs: float):
    from librosa.core import pyin
    return pyin(x, fmin=50, fmax=2000, sr=fs)

def time_stretch(x, rate: float):
    from librosa.effects import time_stretch
    return time_stretch(x, rate=rate)

def pitch_shift(x, fs: float, steps: float):
    from librosa.effects import pitch_shift
    return pitch_shift(x, sr=fs, n_steps=steps)

def harmonic_percussive_split(x):
    from librosa.effects import hpss
    return hpss(x)

def griffin_lim(S, n_iter: int = 32):
    from librosa.core import griffinlim
    return griffinlim(S, n_iter=n_iter)

def stft(x, n_fft: int = 2048, hop: int = 512):
    from librosa.core import stft
    return stft(x, n_fft=n_fft, hop_length=hop)

def istft(S, hop: int = 512):
    from librosa.core import istft
    return istft(S, hop_length=hop)

def power_to_db(S):
    from librosa.core import power_to_db
    return power_to_db(S)

def db_to_power(S):
    from librosa.core import db_to_power
    return db_to_power(S)

def amplitude_to_db(S):
    from librosa.core import amplitude_to_db
    return amplitude_to_db(S)

def frames_to_time(frames, fs: float, hop: int = 512):
    from librosa.core import frames_to_time
    return frames_to_time(frames, sr=fs, hop_length=hop)

def time_to_frames(t, fs: float, hop: int = 512):
    from librosa.core import time_to_frames
    return time_to_frames(t, sr=fs, hop_length=hop)

def load_audio(path: str, fs: float | None = None):
    from librosa.core import load
    return load(path, sr=fs)

def write_audio(path: str, x, fs: float):
    import soundfile as sf
    sf.write(path, x, fs)

def read_audio(path: str):
    import soundfile as sf
    return sf.read(path)

def spectral_centroid(x, fs: float):
    from librosa.feature import spectral_centroid
    return spectral_centroid(y=x, sr=fs)

def spectral_bandwidth(x, fs: float):
    from librosa.feature import spectral_bandwidth
    return spectral_bandwidth(y=x, sr=fs)

def spectral_rolloff(x, fs: float):
    from librosa.feature import spectral_rolloff
    return spectral_rolloff(y=x, sr=fs)

def zero_crossing_rate(x):
    from librosa.feature import zero_crossing_rate
    return zero_crossing_rate(x)

def tonnetz(x, fs: float):
    from librosa.feature import tonnetz
    return tonnetz(y=x, sr=fs)

def delta(features, order: int = 1):
    from librosa.feature import delta
    return delta(features, order=order)

def stack_memory(features, n_steps: int = 2):
    from librosa.feature import stack_memory
    return stack_memory(features, n_steps=n_steps)

def recurrence_matrix(features):
    from librosa.segment import recurrence_matrix
    return recurrence_matrix(features)

def agglomerative_segment(features, k: int):
    from librosa.segment import agglomerative
    return agglomerative(features, k)

def viterbi_decode(prob, transition):
```

```
```

```
```

```
```

```
(define (main)
  (display "Hello, world!")
  (newline))
(main)
```

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
log() { printf '[%s] %s\n' "$(date +%H:%M:%S)" "$*" >&2; }
die() { log "ERROR: $*"; exit 1; }
[[ $# -ge 1 ]] || die "usage: $0 <name>"
name="$1"
log "hello, ${name}"
```

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    int n = argc > 1 ? atoi(argv[1]) : 10;
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        printf("%ld ", a);
        long t = a + b;
        a = b;
        b = t;
    }
    putchar('\n');
    return 0;
}
```

```
#include <iostream>
#include <vector>
#include <numeric>
int main() {
    std::vector<int> v{1, 2, 3, 4, 5};
    int sum = std::accumulate(v.begin(), v.end(), 0);
    std::cout << "sum = " << sum << '\n';
    return 0;
}
```

```
(defn fizzbuzz [n]
  (cond
    (zero? (mod n 15)) "FizzBuzz"
    (zero? (mod n 3))  "Fizz"
    (zero? (mod n 5))  "Buzz"
    :else              n))
(doseq [i (range 1 101)]
  (println (fizzbuzz i)))
```

```
defmodule Greeter do
  def hello(name) when is_binary(name) do
    "Hello, #{name}!"
  end
  def hello(_), do: raise ArgumentError, "name must be a string"
end
IO.puts Greeter.hello("world")
```

```
package main
import (
    "fmt"
    "sync"
)
func worker(id int, jobs <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for j := range jobs {
        fmt.Printf("worker %d processing job %d\n", id, j)
    }
}
func main() {
    jobs := make(chan int, 100)
    var wg sync.WaitGroup
    for w := 1; w <= 3; w++ {
        wg.Add(1)
        go worker(w, jobs, &wg)
    }
    for j := 1; j <= 9; j++ {
        jobs <- j
    }
    close(jobs)
    wg.Wait()
}
```

```
import Data.List (sort)
quicksort :: Ord a => [a] -> [a]
quicksort []     = []
quicksort (p:xs) = quicksort lesser ++ [p] ++ quicksort greater
  where
    lesser  = filter (< p) xs
    greater = filter (>= p) xs
main :: IO ()
main = print (quicksort [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5])
```

```
#include <iostream>
#include <vector>
#include <numeric>
int main() {
    std::vector<int> v{1, 2, 3, 4, 5};
    int sum = std::accumulate(v.begin(), v.end(), 0);
    std::cout << "sum = " << sum << '\n';
    return 0;
}
```

```
(defn fizzbuzz [n]
  (cond
    (zero? (mod n 15)) "FizzBuzz"
    (zero? (mod n 3))  "Fizz"
    (zero? (mod n 5))  "Buzz"
    :else              n))
(doseq [i (range 1 101)]
  (println (fizzbuzz i)))
```

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    int n = argc > 1 ? atoi(argv[1]) : 10;
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        printf("%ld ", a);
        long t = a + b;
        a = b;
        b = t;
    }
    putchar('\n');
    return 0;
}
```

```
(define (main)
  (display "Hello, world!")
  (newline))
(main)
```

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
log() { printf '[%s] %s\n' "$(date +%H:%M:%S)" "$*" >&2; }
die() { log "ERROR: $*"; exit 1; }
[[ $# -ge 1 ]] || die "usage: $0 <name>"
name="$1"
log "hello, ${name}"
```

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
    int n = argc > 1 ? atoi(argv[1]) : 10;
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        printf("%ld ", a);
        long t = a + b;
        a = b;
        b = t;
    }
    putchar('\n');
    return 0;
}
```

```
# ============================================================================
# Auto-generated Terraform configuration
# Provider: aws ~> 5.0
# Region: us-east-1
# ============================================================================

terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "~> 3.5"
    }
    tls = {
      source  = "hashicorp/tls"
      version = "~> 4.0"
    }
  }

  backend "s3" {
    bucket         = "terraform-state-prod-a1b2c3"
    key            = "infrastructure/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-lock"
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Environment = var.environment
      Project     = var.project_name
      ManagedBy   = "terraform"
      Owner       = "platform-team"
      CostCenter  = "engineering"
    }
  }
}

provider "aws" {
  alias  = "us_west_2"
  region = "us-west-2"
}

# ----------------------------------------------------------------------------
# Variables
# ----------------------------------------------------------------------------

variable "aws_region" {
  description = "Primary AWS region for resources"
  type        = string
  default     = "us-east-1"
}

variable "environment" {
  description = "Deployment environment (dev, staging, prod)"
  type        = string
  default     = "prod"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "project_name" {
  description = "Name of the project"
  type        = string
  default     = "acme-platform"
}

variable "vpc_cidr" {
  description = "CIDR block for the VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "availability_zones" {
  description = "List of availability zones"
  type        = list(string)
  default     = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

variable "db_instance_class" {
  description = "RDS instance class"
  type        = string
  default     = "db.r6g.xlarge"
}

variable "db_password" {
  description = "Master password for RDS"
  type        = string
  sensitive   = true
}

variable "enable_multi_az" {
  description = "Enable Multi-AZ deployment for RDS"
  type        = bool
  default     = true
}

variable "eks_node_instance_types" {
  description = "Instance types for EKS node group"
  type        = list(string)
  default     = ["m6i.large", "m6i.xlarge"]
}

variable "eks_desired_nodes" {
  description = "Desired number of EKS nodes"
  type        = number
  default     = 3
}

variable "eks_min_nodes" {
  description = "Minimum number of EKS nodes"
  type        = number
  default     = 2
}

variable "eks_max_nodes" {
  description = "Maximum number of EKS nodes"
  type        = number
  default     = 10
}

variable "tags" {
  description = "Additional tags to apply to resources"
  type        = map(string)
  default     = {}
}

# ----------------------------------------------------------------------------
# Locals
# ----------------------------------------------------------------------------

locals {
  name_prefix = "${var.project_name}-${var.environment}"

  common_tags = merge(var.tags, {
    Environment = var.environment
    Project     = var.project_name
  })

  public_subnet_cidrs  = [for i, az in var.availability_zones : cidrsubnet(var.vpc_cidr, 8, i)]
  private_subnet_cidrs = [for i, az in var.availability_zones : cidrsubnet(var.vpc_cidr, 8, i + 100)]
}

# ----------------------------------------------------------------------------
# VPC
# ----------------------------------------------------------------------------

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-vpc"
  })
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-igw"
  })
}

resource "aws_subnet" "public" {
  count                   = length(var.availability_zones)
  vpc_id                  = aws_vpc.main.id
  cidr_block              = local.public_subnet_cidrs[count.index]
  availability_zone       = var.availability_zones[count.index]
  map_public_ip_on_launch = true

  tags = merge(local.common_tags, {
    Name                     = "${local.name_prefix}-public-${var.availability_zones[count.index]}"
    "kubernetes.io/role/elb" = "1"
  })
}

resource "aws_subnet" "private" {
  count             = length(var.availability_zones)
  vpc_id            = aws_vpc.main.id
  cidr_block        = local.private_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]

  tags = merge(local.common_tags, {
    Name                              = "${local.name_prefix}-private-${var.availability_zones[count.index]}"
    "kubernetes.io/role/internal-elb" = "1"
  })
}

resource "aws_eip" "nat" {
  count  = length(var.availability_zones)
  domain = "vpc"

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-nat-eip-${count.index}"
  })
}

resource "aws_nat_gateway" "main" {
  count         = length(var.availability_zones)
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-nat-${count.index}"
  })

  depends_on = [aws_internet_gateway.main]
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-public-rt"
  })
}

resource "aws_route_table" "private" {
  count  = length(var.availability_zones)
  vpc_id = aws_vpc.main.id

  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main[count.index].id
  }

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-private-rt-${count.index}"
  })
}

resource "aws_route_table_association" "public" {
  count          = length(var.availability_zones)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  count          = length(var.availability_zones)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private[count.index].id
}

# ----------------------------------------------------------------------------
# Security Groups
# ----------------------------------------------------------------------------

resource "aws_security_group" "rds" {
  name_prefix = "${local.name_prefix}-rds-"
  vpc_id      = aws_vpc.main.id
  description = "Security group for RDS PostgreSQL"

  ingress {
    from_port   = 5432
    to_port     = 5432
    protocol    = "tcp"
    cidr_blocks = [var.vpc_cidr]
    description = "PostgreSQL from VPC"
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-rds-sg"
  })

  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_security_group" "eks_nodes" {
  name_prefix = "${local.name_prefix}-eks-nodes-"
  vpc_id      = aws_vpc.main.id
  description = "Security group for EKS worker nodes"

  ingress {
    from_port   = 0
    to_port     = 65535
    protocol    = "tcp"
    self        = true
    description = "Node to node communication"
  }

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = [var.vpc_cidr]
    description = "HTTPS from VPC"
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-eks-nodes-sg"
  })

  lifecycle {
    create_before_destroy = true
  }
}

# ----------------------------------------------------------------------------
# RDS PostgreSQL
# ----------------------------------------------------------------------------

resource "aws_db_subnet_group" "main" {
  name       = "${local.name_prefix}-db-subnet-group"
  subnet_ids = aws_subnet.private[*].id

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-db-subnet-group"
  })
}

resource "aws_db_parameter_group" "main" {
  name_prefix = "${local.name_prefix}-pg15-"
  family      = "postgres15"
  description = "Custom parameter group for PostgreSQL 15"

  parameter {
    name  = "log_connections"
    value = "1"
  }

  parameter {
    name  = "log_disconnections"
    value = "1"
  }

  parameter {
    name  = "log_min_duration_statement"
    value = "500"
  }

  parameter {
    name  = "shared_preload_libraries"
    value = "pg_stat_statements"
  }

  tags = local.common_tags

  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_db_instance" "main" {
  identifier_prefix = "${local.name_prefix}-db-"
  engine            = "postgres"
  engine_version    = "15.4"
  instance_class    = var.db_instance_class
  allocated_storage = 100
  max_allocated_storage = 500
  storage_type      = "gp3"
  storage_encrypted = true

  db_name  = "appdb"
  username = "dbadmin"
  password = var.db_password

  db_subnet_group_name   = aws_db_subnet_group.main.name
  parameter_group_name   = aws_db_parameter_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  multi_az                = var.enable_multi_az
  backup_retention_period = 7
  backup_window           = "03:00-04:00"
  maintenance_window      = "Mon:04:00-Mon:05:00"

  skip_final_snapshot       = false
  final_snapshot_identifier = "${local.name_prefix}-db-final-snapshot"
  deletion_protection       = true

  performance_insights_enabled = true

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-db"
  })
}

# ----------------------------------------------------------------------------
# EKS Cluster
# ----------------------------------------------------------------------------

resource "aws_iam_role" "eks_cluster" {
  name_prefix = "${local.name_prefix}-eks-cluster-"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "eks.amazonaws.com"
      }
    }]
  })

  tags = local.common_tags
}

resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = aws_iam_role.eks_cluster.name
}

resource "aws_iam_role_policy_attachment" "eks_vpc_resource_controller" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSVPCResourceController"
  role       = aws_iam_role.eks_cluster.name
}

resource "aws_eks_cluster" "main" {
  name     = "${local.name_prefix}-eks"
  role_arn = aws_iam_role.eks_cluster.arn
  version  = "1.28"

  vpc_config {
    subnet_ids              = concat(aws_subnet.public[*].id, aws_subnet.private[*].id)
    endpoint_private_access = true
    endpoint_public_access  = true
    public_access_cidrs     = ["0.0.0.0/0"]
  }

  enabled_cluster_log_types = ["api", "audit", "authenticator", "controllerManager", "scheduler"]

  tags = local.common_tags

  depends_on = [
    aws_iam_role_policy_attachment.eks_cluster_policy,
    aws_iam_role_policy_attachment.eks_vpc_resource_controller,
  ]
}

resource "aws_iam_role" "eks_node_group" {
  name_prefix = "${local.name_prefix}-eks-node-"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })

  tags = local.common_tags
}

resource "aws_iam_role_policy_attachment" "eks_worker_node_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
  role       = aws_iam_role.eks_node_group.name
}

resource "aws_iam_role_policy_attachment" "eks_cni_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
  role       = aws_iam_role.eks_node_group.name
}

resource "aws_iam_role_policy_attachment" "ecr_read_only" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
  role       = aws_iam_role.eks_node_group.name
}

resource "aws_eks_node_group" "main" {
  cluster_name    = aws_eks_cluster.main.name
  node_group_name = "${local.name_prefix}-ng"
  node_role_arn   = aws_iam_role.eks_node_group.arn
  subnet_ids      = aws_subnet.private[*].id
  instance_types  = var.eks_node_instance_types

  scaling_config {
    desired_size = var.eks_desired_nodes
    min_size     = var.eks_min_nodes
    max_size     = var.eks_max_nodes
  }

  update_config {
    max_unavailable = 1
  }

  tags = local.common_tags

  depends_on = [
    aws_iam_role_policy_attachment.eks_worker_node_policy,
    aws_iam_role_policy_attachment.eks_cni_policy,
    aws_iam_role_policy_attachment.ecr_read_only,
  ]
}

# ----------------------------------------------------------------------------
# ECR Repository
# ----------------------------------------------------------------------------

resource "aws_ecr_repository" "app" {
  name                 = "${local.name_prefix}-app"
  image_tag_mutability = "MUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "AES256"
  }

  tags = local.common_tags
}

resource "aws_ecr_lifecycle_policy" "app" {
  repository = aws_ecr_repository.app.name

  policy = jsonencode({
    rules = [{
      rulePriority = 1
      description  = "Keep last 30 images"
      selection = {
        tagStatus   = "any"
        countType   = "imageCountMoreThan"
        countNumber = 30
      }
      action = {
        type = "expire"
      }
    }]
  })
}

# ----------------------------------------------------------------------------
# Outputs
# ----------------------------------------------------------------------------

output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "IDs of public subnets"
  value       = aws_subnet.public[*].id
}

output "private_subnet_ids" {
  description = "IDs of private subnets"
  value       = aws_subnet.private[*].id
}

output "rds_endpoint" {
  description = "RDS instance endpoint"
  value       = aws_db_instance.main.endpoint
}

output "rds_port" {
  description = "RDS instance port"
  value       = aws_db_instance.main.port
}

output "eks_cluster_name" {
  description = "EKS cluster name"
  value       = aws_eks_cluster.main.name
}

output "eks_cluster_endpoint" {
  description = "EKS cluster API endpoint"
  value       = aws_eks_cluster.main.endpoint
}

output "eks_cluster_certificate_authority" {
  description = "EKS cluster CA certificate"
  value       = aws_eks_cluster.main.certificate_authority[0].data
}

output "ecr_repository_url" {
  description = "ECR repository URL"
  value       = aws_ecr_repository.app.repository_url
}

output "nat_gateway_ips" {
  description = "Public IPs of NAT gateways"
  value       = aws_eip.nat[*].public_ip
}
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```

```
```
```
```

```
```

---

```
```

---

```
```

```
# ansible_ssh_user = root  # Uncomment to use root user
# ansible_ssh_pass = secret  # Uncomment to specify password (not recommended)
# ansible_sudo_pass = secret  # Uncomment to specify sudo password (not recommended)
# ansible_sudo = True  # Uncomment to force sudo to be used
# ansible_port = 1234  # Uncomment to override the default port
# ansible_ssh_host = 10.42.0.1  # Uncomment to override the default host
# ansible_connection = ssh  # Uncomment to override the default connection type
# ansible_ssh_private_key_file = /path/to/file  # Uncomment to override the default key
# ansible_shell_type = csh  # Uncomment to override the default shell type
# ansible_python_interpreter = /usr/bin/python3  # Uncomment to override the default python interpreter
# ansible_shell_type=zsh
# ansible_ssh_user=mpdehaan
# ansible_connection=ssh
# ansible_ssh_pass="password"
# ansible_ssh_private_key_file=/path/to/file.ssh.key
# ansible_shell_executable=/bin/zsh
# ansible_sudo_exe=/bin/mysudo
# ansible_sftp_extra_args="-vvv"
# ansible_scp_extra_args="-vvv"
# ansible_ssh_extra_args="-vvv"
# ansible_ssh_pipelining=True
# ansible_ssh_executable=/usr/bin/ssh
# ansible_ssh_common_args="-o ProxyCommand='ssh -W %h:%p -q user@gateway.example.com'"
# ansible_docker_extra_args="--tlsverify --tlscacert=/path/to/ca.pem --tlscert=/path/to/client-cert.pem --tlskey=/path/to/client-key.pem -H=tcp://localhost:4000"
# ansible_become=yes
# ansible_become_method=sudo
# ansible_become_user=root
# ansible_become_pass=password
# ansible_become_exe=/usr/bin/sudo
# ansible_become_flags=-H -S -n
# ansible_network_os=ios
# ansible_network_os=junos
# ansible_network_os=vyos
# ansible_network_os=nxos
# ansible_network_os=eos
# ansible_network_os=asa
# ansible_network_os=aruba
# ansible_network_os=ironware
# ansible_network_os=routeros
# ansible_network_os=enos
# ansible_network_os=dellos9
# ansible_network_os=dellos10
# ansible_network_os=dellos6
# ansible_network_os=voss
# ansible_network_os=slxos
# ansible_network_os=netvisor
# ansible_network_os=sros
# ansible_network_os=layer3
# ansible_network_os=layer2
# ansible_network_os=cumulus
# ansible_network_os=ce
# ansible_network_os=pluribus
# ansible_network_os=fortios
# ansible_network_os=asa
# ansible_network_os=exos
# ansible_network_os=ftd
# ansible_network_os=netscaler
# ansible_network_os=panos
# ansible_network_os=vyos
# ansible_network_os=sonic
# ansible_network_os=openswitch
# ansible_network_os=icx
# ansible_network_os=ironware
```

```
```

```
                    interpreter_environment_merging_test>
                        <test_name="merging_environments_with_conflicting_bindings" />
                    </interpreter_environment_merging_test>
                </environment_merging_tests>
                <!-- Function call tests -->
                <function_call_tests>
                    <!-- Simple function call -->
                    <simple_function_call>
                        <input>
                            <code_to_execute>
                                (define (add x y) (+ x y))
                                (add 3 4)
                            </code_to_execute>
                        </input>
                        <expected_output>7</expected_output>
                        <environment_state>
                            <binding name="add" type="function" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                        </environment_state>
                    </simple_function_call>
                    <!-- Function call with closure -->
                    <closure_function_call>
                        <input>
                            <code_to_execute>
                                (define (make-adder n)
                                  (lambda (x) (+ x n)))
                                (define add5 (make-adder 5))
                                (add5 10)
                            </code_to_execute>
                        </input>
                        <expected_output>15</expected_output>
                        <environment_state>
                            <binding name="make-adder" type="function" />
                            <binding name="add5" type="function" />
                            <binding name="n" type="captured" />
                            <binding name="x" type="local" />
                        </environment_state>
                    </closure_function_call>
                    <!-- Recursive function call -->
                    <recursive_function_call>
                        <input>
                            <code_to_execute>
                                (define (factorial n)
                                  (if (= n 0)
                                      1
                                      (* n (factorial (- n 1)))))
                                (factorial 5)
                            </code_to_execute>
                        </input>
                        <expected_output>120</expected_output>
                        <environment_state>
                            <binding name="factorial" type="function" />
                            <binding name="n" type="local" />
                        </environment_state>
                    </recursive_function_call>
                    <!-- Function call with multiple arguments -->
                    <multiple_arguments_function_call>
                        <input>
                            <code_to_execute>
                                (define (sum-of-squares x y z)
                                  (+ (* x x) (* y y) (* z z)))
                                (sum-of-squares 1 2 3)
                            </code_to_execute>
                        </input>
                        <expected_output>14</expected_output>
                        <environment_state>
                            <binding name="sum-of-squares" type="function" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                            <binding name="z" type="local" />
                        </environment_state>
                    </multiple_arguments_function_call>
                    <!-- Function call with variable arguments -->
                    <variable_arguments_function_call>
                        <input>
                            <code_to_execute>
                                (define (sum . numbers)
                                  (apply + numbers))
                                (sum 1 2 3 4 5)
                            </code_to_execute>
                        </input>
                        <expected_output>15</expected_output>
                        <environment_state>
                            <binding name="sum" type="function" />
                            <binding name="numbers" type="local" />
                        </environment_state>
                    </variable_arguments_function_call>
                    <!-- Function call with higher-order function -->
                    <higher_order_function_call>
                        <input>
                            <code_to_execute>
                                (define (apply-twice f x)
                                  (f (f x)))
                                (define (add1 x) (+ x 1))
                                (apply-twice add1 5)
                            </code_to_execute>
                        </input>
                        <expected_output>7</expected_output>
                        <environment_state>
                            <binding name="apply-twice" type="function" />
                            <binding name="f" type="local" />
                            <binding name="x" type="local" />
                            <binding name="add1" type="function" />
                        </environment_state>
                    </higher_order_function_call>
                    <!-- Function call with tail recursion -->
                    <tail_recursive_function_call>
                        <input>
                            <code_to_execute>
                                (define (factorial-tail n acc)
                                  (if (= n 0)
                                      acc
                                      (factorial-tail (- n 1) (* n acc))))
                                (factorial-tail 5 1)
                            </code_to_execute>
                        </input>
                        <expected_output>120</expected_output>
                        <environment_state>
                            <binding name="factorial-tail" type="function" />
                            <binding name="n" type="local" />
                            <binding name="acc" type="local" />
                        </environment_state>
                    </tail_recursive_function_call>
                    <!-- Function call with mutual recursion -->
                    <mutual_recursive_function_call>
                        <input>
                            <code_to_execute>
                                (define (even? n)
                                  (if (= n 0)
                                      #t
                                      (odd? (- n 1))))
                                (define (odd? n)
                                  (if (= n 0)
                                      #f
                                      (even? (- n 1))))
                                (even? 10)
                            </code_to_execute>
                        </input>
                        <expected_output>#t</expected_output>
                        <environment_state>
                            <binding name="even?" type="function" />
                            <binding name="odd?" type="function" />
                            <binding name="n" type="local" />
                        </environment_state>
                    </mutual_recursive_function_call>
                    <!-- Function call with continuation -->
                    <continuation_function_call>
                        <input>
                            <code_to_execute>
                                (define (call-with-current-continuation f)
                                  (f (lambda (x) x)))
                                (call-with-current-continuation (lambda (k) (k 42)))
                            </code_to_execute>
                        </input>
                        <expected_output>42</expected_output>
                        <environment_state>
                            <binding name="call-with-current-continuation" type="function" />
                            <binding name="f" type="local" />
                            <binding name="k" type="local" />
                        </environment_state>
                    </continuation_function_call>
                    <!-- Function call with dynamic binding -->
                    <dynamic_binding_function_call>
                        <input>
                            <code_to_execute>
                                (define x 10)
                                (define (get-x) x)
                                (define (set-x! new-x) (set! x new-x))
                                (set-x! 20)
                                (get-x)
                            </code_to_execute>
                        </input>
                        <expected_output>20</expected_output>
                        <environment_state>
                            <binding name="x" type="global" />
                            <binding name="get-x" type="function" />
                            <binding name="set-x!" type="function" />
                            <binding name="new-x" type="local" />
                        </environment_state>
                    </dynamic_binding_function_call>
                    <!-- Function call with error handling -->
                    <error_handling_function_call>
                        <input>
                            <code_to_execute>
                                (define (safe-divide x y)
                                  (if (= y 0)
                                      (error "Division by zero")
                                      (/ x y)))
                                (safe-divide 10 0)
                            </code_to_execute>
                        </input>
                        <expected_output>Error: Division by zero</expected_output>
                        <environment_state>
                            <binding name="safe-divide" type="function" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                        </environment_state>
                    </error_handling_function_call>
                    <!-- Function call with macro expansion -->
                    <macro_expansion_function_call>
                        <input>
                            <code_to_execute>
                                (define-syntax when
                                  (syntax-rules ()
                                    ((when test body ...)
                                     (if test (begin body ...)))))
                                (when (> 5 3) (display "Hello") (newline))
                            </code_to_execute>
                        </input>
                        <expected_output>Hello
                        </expected_output>
                        <environment_state>
                            <binding name="when" type="macro" />
                            <binding name="test" type="local" />
                            <binding name="body" type="local" />
                        </environment_state>
                    </macro_expansion_function_call>
                    <!-- Function call with lazy evaluation -->
                    <lazy_evaluation_function_call>
                        <input>
                            <code_to_execute>
                                (define (delay expr)
                                  (lambda () expr))
                                (define (force promise)
                                  (promise))
                                (define p (delay (+ 1 2)))
                                (force p)
                            </code_to_execute>
                        </input>
                        <expected_output>3</expected_output>
                        <environment_state>
                            <binding name="delay" type="function" />
                            <binding name="force" type="function" />
                            <binding name="promise" type="local" />
                            <binding name="p" type="global" />
                        </environment_state>
                    </lazy_evaluation_function_call>
                    <!-- Function call with memoization -->
                    <memoization_function_call>
                        <input>
                            <code_to_execute>
                                (define (memoize f)
                                  (let ((cache (make-hash-table)))
                                    (lambda (x)
                                      (or (hash-table-ref cache x)
                                          (let ((result (f x)))
                                            (hash-table-set! cache x result)
                                            result)))))
                                (define (fib n)
                                  (if (< n 2)
                                      n
                                      (+ (fib (- n 1)) (fib (- n 2)))))
                                (define memo-fib (memoize fib))
                                (memo-fib 10)
                            </code_to_execute>
                        </input>
                        <expected_output>55</expected_output>
                        <environment_state>
                            <binding name="memoize" type="function" />
                            <binding name="cache" type="local" />
                            <binding name="fib" type="function" />
                            <binding name="memo-fib" type="function" />
                        </environment_state>
                    </memoization_function_call>
                    <!-- Function call with pattern matching -->
                    <pattern_matching_function_call>
                        <input>
                            <code_to_execute>
                                (define (match-list lst)
                                  (cond ((null? lst) 'empty)
                                        ((pair? lst) 'pair)
                                        (else 'unknown)))
                                (match-list '(1 2 3))
                            </code_to_execute>
                        </input>
                        <expected_output>pair</expected_output>
                        <environment_state>
                            <binding name="match-list" type="function" />
                            <binding name="lst" type="local" />
                        </environment_state>
                    </pattern_matching_function_call>
                    <!-- Function call with object-oriented programming -->
                    <object_oriented_function_call>
                        <input>
                            <code_to_execute>
                                (define (make-point x y)
                                  (lambda (msg)
                                    (cond ((eq? msg 'x) x)
                                          ((eq? msg 'y) y)
                                          ((eq? msg 'move)
                                           (lambda (dx dy)
                                             (make-point (+ x dx) (+ y dy))))
                                          (else (error "Unknown message")))))
                                (define p1 (make-point 3 4))
                                (p1 'x)
                            </code_to_execute>
                        </input>
                        <expected_output>3</expected_output>
                        <environment_state>
                            <binding name="make-point" type="function" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                            <binding name="msg" type="local" />
                            <binding name="p1" type="global" />
                        </environment_state>
                    </object_oriented_function_call>
                    <!-- Function call with concurrency -->
                    <concurrency_function_call>
                        <input>
                            <code_to_execute>
                                (define (spawn thunk)
                                  (thread-start! (make-thread thunk)))
                                (define (worker)
                                  (display "Working...")
                                  (newline))
                                (spawn worker)
                                (thread-sleep! 1)
                            </code_to_execute>
                        </input>
                        <expected_output>Working...
                        </expected_output>
                        <environment_state>
                            <binding name="spawn" type="function" />
                            <binding name="thunk" type="local" />
                            <binding name="worker" type="function" />
                        </environment_state>
                    </concurrency_function_call>
                    <!-- Function call with foreign function interface -->
                    <foreign_function_call>
                        <input>
                            <code_to_execute>
                                (define-external (add-numbers (int x) (int y)) int
                                  (+ x y))
                                (add-numbers 3 4)
                            </code_to_execute>
                        </input>
                        <expected_output>7</expected_output>
                        <environment_state>
                            <binding name="add-numbers" type="function" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                        </environment_state>
                    </foreign_function_call>
                    <!-- Function call with reflection -->
                    <reflection_function_call>
                        <input>
                            <code_to_execute>
                                (define (get-procedure-arity proc)
                                  (##procedure-arity proc))
                                (get-procedure-arity +)
                            </code_to_execute>
                        </input>
                        <expected_output>-1</expected_output>
                        <environment_state>
                            <binding name="get-procedure-arity" type="function" />
                            <binding name="proc" type="local" />
                        </environment_state>
                    </reflection_function_call>
                    <!-- Function call with serialization -->
                    <serialization_function_call>
                        <input>
                            <code_to_execute>
                                (define (serialize obj)
                                  (with-output-to-string
                                    (lambda () (write obj))))
                                (define (deserialize str)
                                  (with-input-from-string str read))
                                (deserialize (serialize '(1 2 3)))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3)</expected_output>
                        <environment_state>
                            <binding name="serialize" type="function" />
                            <binding name="deserialize" type="function" />
                            <binding name="obj" type="local" />
                            <binding name="str" type="local" />
                        </environment_state>
                    </serialization_function_call>
                    <!-- Function call with debugging -->
                    <debugging_function_call>
                        <input>
                            <code_to_execute>
                                (define (trace-calls f)
                                  (lambda args
                                    (display "Calling with: ")
                                    (write args)
                                    (newline)
                                    (let ((result (apply f args)))
                                      (display "Result: ")
                                      (write result)
                                      (newline)
                                      result)))
                                (define (add x y) (+ x y))
                                (define traced-add (trace-calls add))
                                (traced-add 3 4)
                            </code_to_execute>
                        </input>
                        <expected_output>Calling with: (3 4)
                        Result: 7
                        7</expected_output>
                        <environment_state>
                            <binding name="trace-calls" type="function" />
                            <binding name="f" type="local" />
                            <binding name="args" type="local" />
                            <binding name="result" type="local" />
                            <binding name="add" type="function" />
                            <binding name="traced-add" type="function" />
                        </environment_state>
                    </debugging_function_call>
                    <!-- Function call with profiling -->
                    <profiling_function_call>
                        <input>
                            <code_to_execute>
                                (define (time-it thunk)
                                  (let ((start (current-time)))
                                    (thunk)
                                    (let ((end (current-time)))
                                      (- end start))))
                                (define (slow-function)
                                  (sleep 1))
                                (time-it slow-function)
                            </code_to_execute>
                        </input>
                        <expected_output>1</expected_output>
                        <environment_state>
                            <binding name="time-it" type="function" />
                            <binding name="thunk" type="local" />
                            <binding name="start" type="local" />
                            <binding name="end" type="local" />
                            <binding name="slow-function" type="function" />
                        </environment_state>
                    </profiling_function_call>
                    <!-- Function call with testing framework -->
                    <testing_framework_function_call>
                        <input>
                            <code_to_execute>
                                (define (assert-equal expected actual)
                                  (if (equal? expected actual)
                                      (display "Test passed")
                                      (display "Test failed")))
                                (assert-equal 4 (+ 2 2))
                            </code_to_execute>
                        </input>
                        <expected_output>Test passed</expected_output>
                        <environment_state>
                            <binding name="assert-equal" type="function" />
                            <binding name="expected" type="local" />
                            <binding name="actual" type="local" />
                        </environment_state>
                    </testing_framework_function_call>
                    <!-- Function call with documentation -->
                    <documentation_function_call>
                        <input>
                            <code_to_execute>
                                (define (square x)
                                  "Returns the square of x."
                                  (* x x))
                                (square 5)
                            </code_to_execute>
                        </input>
                        <expected_output>25</expected_output>
                        <environment_state>
                            <binding name="square" type="function" />
                            <binding name="x" type="local" />
                        </environment_state>
                    </documentation_function_call>
                    <!-- Function call with type checking -->
                    <type_checking_function_call>
                        <input>
                            <code_to_execute>
                                (define (add-numbers x y)
                                  (if (and (number? x) (number? y))
                                      (+ x y)
                                      (error "Arguments must be numbers")))
                                (add-numbers 3 4)
                            </code_to_execute>
                        </input>
                        <expected_output>7</expected_output>
                        <environment_state>
                            <binding name="add-numbers" type="function" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                        </environment_state>
                    </type_checking_function_call>
                    <!-- Function call with optional arguments -->
                    <optional_arguments_function_call>
                        <input>
                            <code_to_execute>
                                (define (greet name #!optional (greeting "Hello"))
                                  (display greeting)
                                  (display ", ")
                                  (display name)
                                  (newline))
                                (greet "Alice")
                                (greet "Bob" "Hi")
                            </code_to_execute>
                        </input>
                        <expected_output>Hello, Alice
                        Hi, Bob
                        </expected_output>
                        <environment_state>
                            <binding name="greet" type="function" />
                            <binding name="name" type="local" />
                            <binding name="greeting" type="local" />
                        </environment_state>
                    </optional_arguments_function_call>
                    <!-- Function call with keyword arguments -->
                    <keyword_arguments_function_call>
                        <input>
                            <code_to_execute>
                                (define (make-person #!key name age)
                                  (list name age))
                                (make-person name: "Alice" age: 30)
                            </code_to_execute>
                        </input>
                        <expected_output>("Alice" 30)</expected_output>
                        <environment_state>
                            <binding name="make-person" type="function" />
                            <binding name="name" type="local" />
                            <binding name="age" type="local" />
                        </environment_state>
                    </keyword_arguments_function_call>
                    <!-- Function call with destructuring -->
                    <destructuring_function_call>
                        <input>
                            <code_to_execute>
                                (define (swap pair)
                                  (let ((x (car pair))
                                        (y (cdr pair)))
                                    (cons y x)))
                                (swap '(1 . 2))
                            </code_to_execute>
                        </input>
                        <expected_output>(2 . 1)</expected_output>
                        <environment_state>
                            <binding name="swap" type="function" />
                            <binding name="pair" type="local" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                        </environment_state>
                    </destructuring_function_call>
                    <!-- Function call with default arguments -->
                    <default_arguments_function_call>
                        <input>
                            <code_to_execute>
                                (define (power x #!optional (n 2))
                                  (expt x n))
                                (power 3)
                                (power 3 3)
                            </code_to_execute>
                        </input>
                        <expected_output>9
                        27</expected_output>
                        <environment_state>
                            <binding name="power" type="function" />
                            <binding name="x" type="local" />
                            <binding name="n" type="local" />
                        </environment_state>
                    </default_arguments_function_call>
                    <!-- Function call with rest arguments -->
                    <rest_arguments_function_call>
                        <input>
                            <code_to_execute>
                                (define (sum-all . numbers)
                                  (apply + numbers))
                                (sum-all 1 2 3 4 5)
                            </code_to_execute>
                        </input>
                        <expected_output>15</expected_output>
                        <environment_state>
                            <binding name="sum-all" type="function" />
                            <binding name="numbers" type="local" />
                        </environment_state>
                    </rest_arguments_function_call>
                    <!-- Function call with keyword and optional arguments -->
                    <keyword_optional_arguments_function_call>
                        <input>
                            <code_to_execute>
                                (define (make-rectangle #!key (width 1) (height 1))
                                  (* width height))
                                (make-rectangle width: 3 height: 4)
                                (make-rectangle)
                            </code_to_execute>
                        </input>
                        <expected_output>12
                        1</expected_output>
                        <environment_state>
                            <binding name="make-rectangle" type="function" />
                            <binding name="width" type="local" />
                            <binding name="height" type="local" />
                        </environment_state>
                    </keyword_optional_arguments_function_call>
                    <!-- Function call with all argument types -->
                    <all_argument_types_function_call>
                        <input>
                            <code_to_execute>
                                (define (complex-function x #!optional (y 2) #!key (z 3) . rest)
                                  (list x y z rest))
                                (complex-function 1)
                                (complex-function 1 4)
                                (complex-function 1 4 z: 5)
                                (complex-function 1 4 z: 5 6 7 8)
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3 ())
                        (1 4 3 ())
                        (1 4 5 ())
                        (1 4 5 (6 7 8))</expected_output>
                        <environment_state>
                            <binding name="complex-function" type="function" />
                            <binding name="x" type="local" />
                            <binding name="y" type="local" />
                            <binding name="z" type="local" />
                            <binding name="rest" type="local" />
                        </environment_state>
                    </all_argument_types_function_call>
                </function_call_tests>
                <!-- Control flow tests -->
                <control_flow_tests>
                    <!-- Simple if statement -->
                    <simple_if_statement>
                        <input>
                            <code_to_execute>
                                (if (> 5 3) "yes" "no")
                            </code_to_execute>
                        </input>
                        <expected_output>"yes"</expected_output>
                        <environment_state />
                    </simple_if_statement>
                    <!-- If statement with multiple branches -->
                    <multi_branch_if_statement>
                        <input>
                            <code_to_execute>
                                (if (> 5 3) "yes" (if (< 5 3) "no" "maybe"))
                            </code_to_execute>
                        </input>
                        <expected_output>"yes"</expected_output>
                        <environment_state />
                    </multi_branch_if_statement>
                    <!-- Cond statement -->
                    <cond_statement>
                        <input>
                            <code_to_execute>
                                (cond ((> 5 3) "yes")
                                      ((< 5 3) "no")
                                      (else "maybe"))
                            </code_to_execute>
                        </input>
                        <expected_output>"yes"</expected_output>
                        <environment_state />
                    </cond_statement>
                    <!-- Case statement -->
                    <case_statement>
                        <input>
                            <code_to_execute>
                                (case (+ 2 3)
                                  ((1 2) "small")
                                  ((3 4 5) "medium")
                                  (else "large"))
                            </code_to_execute>
                        </input>
                        <expected_output>"medium"</expected_output>
                        <environment_state />
                    </case_statement>
                    <!-- And statement -->
                    <and_statement>
                        <input>
                            <code_to_execute>
                                (and (> 5 3) (< 5 10) "yes")
                            </code_to_execute>
                        </input>
                        <expected_output>"yes"</expected_output>
                        <environment_state />
                    </and_statement>
                    <!-- Or statement -->
                    <or_statement>
                        <input>
                            <code_to_execute>
                                (or (> 5 3) (< 5 10) "yes")
                            </code_to_execute>
                        </input>
                        <expected_output>#t</expected_output>
                        <environment_state />
                    </or_statement>
                    <!-- Not statement -->
                    <not_statement>
                        <input>
                            <code_to_execute>
                                (not (> 5 3))
                            </code_to_execute>
                        </input>
                        <expected_output>#f</expected_output>
                        <environment_state />
                    </not_statement>
                    <!-- When statement -->
                    <when_statement>
                        <input>
                            <code_to_execute>
                                (when (> 5 3) (display "yes") (newline))
                            </code_to_execute>
                        </input>
                        <expected_output>yes
                        </expected_output>
                        <environment_state />
                    </when_statement>
                    <!-- Unless statement -->
                    <unless_statement>
                        <input>
                            <code_to_execute>
                                (unless (< 5 3) (display "yes") (newline))
                            </code_to_execute>
                        </input>
                        <expected_output>yes
                        </expected_output>
                        <environment_state />
                    </unless_statement>
                    <!-- Do loop -->
                    <do_loop>
                        <input>
                            <code_to_execute>
                                (do ((i 0 (+ i 1)))
                                    ((= i 5) i)
                                  (display i)
                                  (newline))
                            </code_to_execute>
                        </input>
                        <expected_output>0
                        1
                        2
                        3
                        4
                        5</expected_output>
                        <environment_state />
                    </do_loop>
                    <!-- While loop -->
                    <while_loop>
                        <input>
                            <code_to_execute>
                                (let ((i 0))
                                  (while (< i 5)
                                    (display i)
                                    (newline)
                                    (set! i (+ i 1))))
                            </code_to_execute>
                        </input>
                        <expected_output>0
                        1
                        2
                        3
                        4</expected_output>
                        <environment_state />
                    </while_loop>
                    <!-- For loop -->
                    <for_loop>
                        <input>
                            <code_to_execute>
                                (for-each (lambda (x)
                                            (display x)
                                            (newline))
                                          '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>1
                        2
                        3
                        4
                        5</expected_output>
                        <environment_state />
                    </for_loop>
                    <!-- Map function -->
                    <map_function>
                        <input>
                            <code_to_execute>
                                (map (lambda (x) (* x x)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 4 9 16 25)</expected_output>
                        <environment_state />
                    </map_function>
                    <!-- Filter function -->
                    <filter_function>
                        <input>
                            <code_to_execute>
                                (filter (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>(4 5)</expected_output>
                        <environment_state />
                    </filter_function>
                    <!-- Fold function -->
                    <fold_function>
                        <input>
                            <code_to_execute>
                                (fold + 0 '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>15</expected_output>
                        <environment_state />
                    </fold_function>
                    <!-- Reduce function -->
                    <reduce_function>
                        <input>
                            <code_to_execute>
                                (reduce + 0 '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>15</expected_output>
                        <environment_state />
                    </reduce_function>
                    <!-- Any function -->
                    <any_function>
                        <input>
                            <code_to_execute>
                                (any (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>#t</expected_output>
                        <environment_state />
                    </any_function>
                    <!-- All function -->
                    <all_function>
                        <input>
                            <code_to_execute>
                                (all (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>#f</expected_output>
                        <environment_state />
                    </all_function>
                    <!-- Find function -->
                    <find_function>
                        <input>
                            <code_to_execute>
                                (find (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>4</expected_output>
                        <environment_state />
                    </find_function>
                    <!-- Partition function -->
                    <partition_function>
                        <input>
                            <code_to_execute>
                                (partition (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((4 5) (1 2 3))</expected_output>
                        <environment_state />
                    </partition_function>
                    <!-- Group by function -->
                    <group_by_function>
                        <input>
                            <code_to_execute>
                                (group-by (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((#t 4 5) (#f 1 2 3))</expected_output>
                        <environment_state />
                    </group_by_function>
                    <!-- Sort function -->
                    <sort_function>
                        <input>
                            <code_to_execute>
                                (sort '(5 3 1 4 2) <)
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3 4 5)</expected_output>
                        <environment_state />
                    </sort_function>
                    <!-- Merge function -->
                    <merge_function>
                        <input>
                            <code_to_execute>
                                (merge '(1 3 5) '(2 4 6) <)
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3 4 5 6)</expected_output>
                        <environment_state />
                    </merge_function>
                    <!-- Zip function -->
                    <zip_function>
                        <input>
                            <code_to_execute>
                                (zip '(1 2 3) '(4 5 6))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 4) (2 5) (3 6))</expected_output>
                        <environment_state />
                    </zip_function>
                    <!-- Unzip function -->
                    <unzip_function>
                        <input>
                            <code_to_execute>
                                (unzip '((1 4) (2 5) (3 6)))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2 3) (4 5 6))</expected_output>
                        <environment_state />
                    </unzip_function>
                    <!-- Flatten function -->
                    <flatten_function>
                        <input>
                            <code_to_execute>
                                (flatten '(1 (2 (3 4) 5) 6))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3 4 5 6)</expected_output>
                        <environment_state />
                    </flatten_function>
                    <!-- Unique function -->
                    <unique_function>
                        <input>
                            <code_to_execute>
                                (unique '(1 2 3 2 1 4 5 4))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3 4 5)</expected_output>
                        <environment_state />
                    </unique_function>
                    <!-- Count function -->
                    <count_function>
                        <input>
                            <code_to_execute>
                                (count (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>2</expected_output>
                        <environment_state />
                    </count_function>
                    <!-- Take function -->
                    <take_function>
                        <input>
                            <code_to_execute>
                                (take 3 '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3)</expected_output>
                        <environment_state />
                    </take_function>
                    <!-- Drop function -->
                    <drop_function>
                        <input>
                            <code_to_execute>
                                (drop 3 '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>(4 5)</expected_output>
                        <environment_state />
                    </drop_function>
                    <!-- Split at function -->
                    <split_at_function>
                        <input>
                            <code_to_execute>
                                (split-at 3 '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2 3) (4 5))</expected_output>
                        <environment_state />
                    </split_at_function>
                    <!-- Span function -->
                    <span_function>
                        <input>
                            <code_to_execute>
                                (span (lambda (x) (< x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2) (3 4 5))</expected_output>
                        <environment_state />
                    </span_function>
                    <!-- Break function -->
                    <break_function>
                        <input>
                            <code_to_execute>
                                (break (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2 3) (4 5))</expected_output>
                        <environment_state />
                    </break_function>
                    <!-- Group adjacent function -->
                    <group_adjacent_function>
                        <input>
                            <code_to_execute>
                                (group-adjacent (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((#f 1 2 3) (#t 4 5))</expected_output>
                        <environment_state />
                    </group_adjacent_function>
                    <!-- Run length encode function -->
                    <run_length_encode_function>
                        <input>
                            <code_to_execute>
                                (run-length-encode '(1 1 2 2 2 3 3 3 3))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2) (2 3) (3 4))</expected_output>
                        <environment_state />
                    </run_length_encode_function>
                    <!-- Run length decode function -->
                    <run_length_decode_function>
                        <input>
                            <code_to_execute>
                                (run-length-decode '((1 2) (2 3) (3 4)))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 1 2 2 2 3 3 3 3)</expected_output>
                        <environment_state />
                    </run_length_decode_function>
                    <!-- Interleave function -->
                    <interleave_function>
                        <input>
                            <code_to_execute>
                                (interleave '(1 2 3) '(4 5 6))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 4 2 5 3 6)</expected_output>
                        <environment_state />
                    </interleave_function>
                    <!-- Partition by function -->
                    <partition_by_function>
                        <input>
                            <code_to_execute>
                                (partition-by (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2 3) (4 5))</expected_output>
                        <environment_state />
                    </partition_by_function>
                    <!-- Group by function -->
                    <group_by_function>
                        <input>
                            <code_to_execute>
                                (group-by (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((#t 4 5) (#f 1 2 3))</expected_output>
                        <environment_state />
                    </group_by_function>
                    <!-- Sort by function -->
                    <sort_by_function>
                        <input>
                            <code_to_execute>
                                (sort-by (lambda (x) (- x)) '(5 3 1 4 2))
                            </code_to_execute>
                        </input>
                        <expected_output>(5 4 3 2 1)</expected_output>
                        <environment_state />
                    </sort_by_function>
                    <!-- Merge by function -->
                    <merge_by_function>
                        <input>
                            <code_to_execute>
                                (merge-by (lambda (x) (- x)) '(5 3 1) '(4 2) <)
                            </code_to_execute>
                        </input>
                        <expected_output>(5 4 3 2 1)</expected_output>
                        <environment_state />
                    </merge_by_function>
                    <!-- Zip with function -->
                    <zip_with_function>
                        <input>
                            <code_to_execute>
                                (zip-with + '(1 2 3) '(4 5 6))
                            </code_to_execute>
                        </input>
                        <expected_output>(5 7 9)</expected_output>
                        <environment_state />
                    </zip_with_function>
                    <!-- Unzip with function -->
                    <unzip_with_function>
                        <input>
                            <code_to_execute>
                                (unzip-with (lambda (x y) (+ x y)) '((1 4) (2 5) (3 6)))
                            </code_to_execute>
                        </input>
                        <expected_output>(5 7 9)</expected_output>
                        <environment_state />
                    </unzip_with_function>
                    <!-- Flatten with function -->
                    <flatten_with_function>
                        <input>
                            <code_to_execute>
                                (flatten-with (lambda (x) (if (list? x) x (list x))) '(1 (2 (3 4) 5) 6))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2 3 4 5 6)</expected_output>
                        <environment_state />
                    </flatten_with_function>
                    <!-- Unique by function -->
                    <unique_by_function>
                        <input>
                            <code_to_execute>
                                (unique-by (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 4)</expected_output>
                        <environment_state />
                    </unique_by_function>
                    <!-- Count by function -->
                    <count_by_function>
                        <input>
                            <code_to_execute>
                                (count-by (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((#t 2) (#f 3))</expected_output>
                        <environment_state />
                    </count_by_function>
                    <!-- Take while function -->
                    <take_while_function>
                        <input>
                            <code_to_execute>
                                (take-while (lambda (x) (< x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 2)</expected_output>
                        <environment_state />
                    </take_while_function>
                    <!-- Drop while function -->
                    <drop_while_function>
                        <input>
                            <code_to_execute>
                                (drop-while (lambda (x) (< x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>(3 4 5)</expected_output>
                        <environment_state />
                    </drop_while_function>
                    <!-- Split with function -->
                    <split_with_function>
                        <input>
                            <code_to_execute>
                                (split-with (lambda (x) (< x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2) (3 4 5))</expected_output>
                        <environment_state />
                    </split_with_function>
                    <!-- Span with function -->
                    <span_with_function>
                        <input>
                            <code_to_execute>
                                (span-with (lambda (x) (< x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2) (3 4 5))</expected_output>
                        <environment_state />
                    </span_with_function>
                    <!-- Break with function -->
                    <break_with_function>
                        <input>
                            <code_to_execute>
                                (break-with (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2 3) (4 5))</expected_output>
                        <environment_state />
                    </break_with_function>
                    <!-- Group adjacent by function -->
                    <group_adjacent_by_function>
                        <input>
                            <code_to_execute>
                                (group-adjacent-by (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((#f 1 2 3) (#t 4 5))</expected_output>
                        <environment_state />
                    </group_adjacent_by_function>
                    <!-- Run length encode by function -->
                    <run_length_encode_by_function>
                        <input>
                            <code_to_execute>
                                (run-length-encode-by (lambda (x) (> x 3)) '(1 1 2 2 2 3 3 3 3))
                            </code_to_execute>
                        </input>
                        <expected_output>((#f 1 2) (#t 3) (#f 2) (#t 3))</expected_output>
                        <environment_state />
                    </run_length_encode_by_function>
                    <!-- Run length decode by function -->
                    <run_length_decode_by_function>
                        <input>
                            <code_to_execute>
                                (run-length-decode-by (lambda (x) (> x 3)) '((#f 1 2) (#t 3) (#f 2) (#t 3)))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 1 2 2 2 3 3 3 3)</expected_output>
                        <environment_state />
                    </run_length_decode_by_function>
                    <!-- Interleave with function -->
                    <interleave_with_function>
                        <input>
                            <code_to_execute>
                                (interleave-with + '(1 2 3) '(4 5 6))
                            </code_to_execute>
                        </input>
                        <expected_output>(1 4 2 5 3 6)</expected_output>
                        <environment_state />
                    </interleave_with_function>
                    <!-- Partition by with function -->
                    <partition_by_with_function>
                        <input>
                            <code_to_execute>
                                (partition-by-with (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((1 2 3) (4 5))</expected_output>
                        <environment_state />
                    </partition_by_with_function>
                    <!-- Group by with function -->
                    <group_by_with_function>
                        <input>
                            <code_to_execute>
                                (group-by-with (lambda (x) (> x 3)) '(1 2 3 4 5))
                            </code_to_execute>
                        </input>
                        <expected_output>((#t 4 5) (#f 1 2 3))</expected_output>
                        <environment_state />
                    </group_by_with_function>
                    <!-- Sort by with function -->
                    <sort_by_with_function>
                        <input>
                            <code_to_execute>
                                (sort-by-with (lambda (x) (- x)) '(5 3 1 4 2))
                            </code_to_execute>
                        </input>
                        <expected_output>(5 4 3 2 1)</expected_output>
                        <environment_state />
                    </sort_by_with_function>
                    <!-- Merge by with function -->
                    <merge_by_with_function>
                        <input>
                            <code_to_execute>
                                (merge-by-with (lambda (x) (- x)) '(5 3 1) '(4 2) <)
                            </code_to_execute>
                        </input>
                        <expected_output>(5 4 3 2 1)</expected_output>
                        <environment_state />
                    </merge_by_with_function>
                    <!-- Zip with with function -->
                    <zip_with_with_function>
                        <input>
                            <code_to_execute>
                
```

```
```